

RECAP for Physical Intelligence VLAs: A Concept Note and Experimental Snapshot

Working Note for `recap_pi`

April 18, 2026

Abstract

This note captures our current understanding of RECAP—*RL with Experience & Corrections via Advantage-conditioned Policies*—as used in Physical Intelligence’s $\pi_{0.6}^*$ system. The core idea is to adapt advantage from reinforcement learning into a form that works naturally for a very large vision-language-action model: estimate advantage offline, binarize it, and feed it back into the policy as a conditioning token. This converts heterogeneous robot data into a stable supervised learning problem while still improving the policy beyond raw imitation. We also summarize the exact iterative training loop, the role of the value function, why the method is particularly well matched to VLAs, and the behavior seen in our local toy experiments.

1 Executive Summary

The “advantage” in RECAP is a VLA-friendly version of the classic RL signal:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

In ordinary actor-critic RL, this quantity drives policy-gradient updates. In RECAP, the same concept is used differently. Instead of backpropagating policy gradients through a giant generative action model, the system:

1. trains a smaller value model to predict future return,
2. computes an estimated advantage for each action in the dataset,
3. binarizes that estimate into a positive/negative indicator,
4. conditions the VLA on that indicator during supervised action training, and
5. requests *positive-advantage* behavior at inference time.

This is the central reason RECAP is compelling for Physical Intelligence style systems. It allows a large VLA to learn from all available robot experience—good demonstrations, mixed autonomous rollouts, and explicit human corrections—without depending on PPO-style optimization on the full model.

2 Advantage from First Principles

2.1 Standard RL Refresher

The value function measures expected future return from a state:

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^T r_t \mid s_0 = s, \pi \right].$$

The state-action value function measures expected return from taking a specific action first:

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{t=0}^T r_t \mid s_0 = s, a_0 = a, \pi \right].$$

Their difference is the advantage:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

Intuitively:

- if $A^\pi(s, a) > 0$, the action did better than the policy’s baseline expectation at that state;
- if $A^\pi(s, a) < 0$, the action was worse than expected.

In locomotion RL or simulated control, this signal is often used directly inside online actor-critic methods such as PPO. For a 5B-parameter VLA with a complex generative action head, that route is much less attractive. RECAP keeps the credit-assignment value of advantage while avoiding direct policy-gradient updates on the full model.

2.2 RECAP’s Key Shift

RECAP turns advantage into *conditioning information*. Instead of asking the VLA to optimize a large RL objective directly, it asks the model to answer a simpler supervised question:

“Given the observation, language command, and a token that says whether this action had positive or negative advantage, what action should be generated?”

That shift is what makes the algorithm fit the VLA setting.

3 RECAP Mechanics

3.1 Task-Agnostic Reward

RECAP uses a simple sparse reward:

$$r_t = \begin{cases} 0 & \text{if the episode ends in success} \\ -C_{\text{fail}} & \text{if the episode ends in failure} \\ -1 & \text{otherwise.} \end{cases}$$

The values are normalized per task to $(-1, 0]$, so 0 corresponds to a perfect outcome and more negative values correspond to slower or failed executions. The reward is intentionally generic: finish quickly and do not fail.

3.2 Value Function

A smaller critic predicts expected return from the current observation o_t and language instruction ℓ . In the $\pi_{0.6}^*$ setup, this is a much smaller vision-language model than the action model itself. It is trained distributionally over binned return targets:

$$\min_{\phi} \mathbb{E}_{\tau} \left[\sum_t H \left(R_t^B(\tau), p_{\phi}(V \mid o_t, \ell) \right) \right].$$

The useful intuition is that the value function acts like a progress estimator or “GPS.” It learns whether the robot is on a trajectory that still looks recoverable and productive, or whether it has drifted toward failure.

3.3 Advantage Estimation

RECAP uses an N -step bootstrapped advantage estimate:

$$A^{\pi}(o_t, a_t) = \left(\sum_{t'=t}^{t+N-1} r_{t'} + V^{\pi}(o_{t+N}) \right) - V^{\pi}(o_t).$$

Interpretation:

- the first term is the realized short-horizon future after taking the action,
- the second term is the baseline expectation before the action.

If the robot made unusually good progress, the advantage is positive. If the action made future success less likely, the advantage is negative.

3.4 Binarization

Rather than feeding a noisy scalar advantage value into the policy, RECAP thresholds it:

$$I_t = \mathbb{I}\{A(o_t, a_t) > \epsilon_{\ell}\}.$$

This gives a simple binary label:

- Advantage: positive
- Advantage: negative

Task-specific thresholds are used so that the positive set has a controlled frequency. During fine-tuning, human correction actions can simply be forced into the positive class.

3.5 Advantage-Conditioned Policy Training

The policy is trained with a supervised objective on all data:

$$\min_{\theta} \mathbb{E} \left[-\log \pi_{\theta}(a_t \mid o_t, \ell) - \alpha \log \pi_{\theta}(a_t \mid I_t, o_t, \ell) \right].$$

The indicator token is randomly dropped some of the time during training. This mirrors classifier-free guidance style conditioning and allows the model to interpolate between unconditional and advantage-conditioned behavior at test time.

At inference, the system simply requests *positive-advantage* behavior. In effect, the policy is nudged toward actions that look more like the high-quality portion of the dataset, even though the model was trained on a broad mixture of behavior.

4 Exact Iterative RECAP Loop

The algorithm can be viewed as an iterative offline RL loop wrapped around a VLA:

```
Require: multi-task demonstration dataset  $\mathcal{D}_{\text{demo}}$ 
1 : Train  $V_{\text{pre}}$  on  $\mathcal{D}_{\text{demo}}$ 
2 : Train  $\pi_{\text{pre}}$  on  $\mathcal{D}_{\text{demo}}$  with advantage conditioning
3 : Initialize task dataset  $\mathcal{D}_{\ell}$ 
4 : Train  $V_{\ell}^0$  from  $V_{\text{pre}}$  on  $\mathcal{D}_{\ell}$ 
5 : Train  $\pi_{\ell}^0$  from  $\pi_{\text{pre}}$  on  $\mathcal{D}_{\ell}$ 
6 : for  $k = 1$  to  $K$  do
7 :   Collect autonomous rollouts and optional human corrections with  $\pi_{\ell}^{k-1}$ 
8 :   Add the new data to  $\mathcal{D}_{\ell}$ 
9 :   Train  $V_{\ell}^k$  from  $V_{\text{pre}}$  on updated  $\mathcal{D}_{\ell}$ 
10 :   Train  $\pi_{\ell}^k$  from  $\pi_{\text{pre}}$  on updated  $\mathcal{D}_{\ell}$ 
11 : end for.
```

Two design choices matter:

- The critic is always reinitialized from the pre-trained value model rather than only warm-started from the previous task critic.
- The policy is always fine-tuned from the pre-trained VLA rather than recursively fine-tuned from the latest on-robot checkpoint.

This keeps the system anchored to the generalist prior while still allowing iterative specialization.

5 Why the Value Function Visualization Matters

The value plots shown in the blog and paper are among the best intuition-building tools for RECAP.

For successful episodes:

- the value rises as the robot completes the meaningful subgoals of the task,
- temporary plateaus often correspond to transport or repositioning,
- small dips can reflect recoverable mistakes such as a slightly bad alignment or grip.

For unsuccessful episodes:

- the value often drops at the exact moment the task becomes difficult or impossible to recover,
- the drop can occur well before the terminal failure label,
- this is what enables long-horizon credit assignment in practical robot tasks.

The important idea is not just that the value predicts eventual success. It identifies *where progress was created or destroyed* inside a long trajectory. That is the signal the advantage estimate turns into action-level supervision.

6 Why RECAP Fits VLAs Better Than Classical Online RL

RECAP is particularly well matched to the Physical Intelligence setting for several reasons:

1. **Heterogeneous data becomes usable.** Demonstrations, autonomous rollouts, and corrections all live in one dataset. The difference is not in whether the data is kept, but in how each action is labeled.
2. **Long-horizon manipulation gets credit assignment.** A bad grasp early in a long task can receive a negative signal even if the failure occurs much later.
3. **Training remains stable for giant generative policies.** The method preserves supervised learning structure instead of relying on high-variance RL updates on the full VLA.
4. **It supports iterative practice.** The robot can deploy, fail, collect corrections, and improve again using the same loop.

This is very different from a typical locomotion setup:

- locomotion often uses dense hand-designed reward terms,
- optimization is online and actor-critic driven,
- the policy may be small enough that PPO-like updates are routine.

RECAP instead uses offline or batched relabeling of behavior data and a conditioned policy model.

7 Toy Interpretation of the Conditioning Trick

The simplest way to understand the method is:

1. train on both good and bad behavior,
2. tag each action with whether it had positive or negative advantage,
3. ask for positive behavior at inference time.

The following sketch captures that idea in minimal form:

Listing 1: Tiny toy sketch of the RECAP conditioning idea.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ToyPolicy(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(3 + 1, 2)

    def forward(self, state, adv_token):
        x = torch.cat([state, adv_token.unsqueeze(1)], dim=1)
        return self.fc(x)
```

```

def generate_data(n=1000):
    states = torch.eye(3)[torch.randint(0, 3, (n,))]
    actions = torch.randint(0, 2, (n,))
    true_good = (torch.rand(n) < 0.8).float()
    advantages = (actions == 1).float() * true_good * 2 - 1
    adv_tokens = (advantages > 0).float()
    return states, actions, adv_tokens

policy = ToyPolicy()
opt = torch.optim.Adam(policy.parameters(), lr=1e-2)
states, actions, adv_tokens = generate_data(5000)

for epoch in range(200):
    logits = policy(states, adv_tokens)
    loss_cond = F.cross_entropy(logits, actions)
    if torch.rand(1) < 0.3:
        logits_uncond = policy(states, torch.zeros_like(adv_tokens))
        loss = loss_cond + F.cross_entropy(logits_uncond, actions)
    else:
        loss = loss_cond
    opt.zero_grad()
    loss.backward()
    opt.step()

with torch.no_grad():
    test_states = torch.eye(3)
    pos_token = torch.ones(3)
    probs = F.softmax(policy(test_states, pos_token), dim=1)
    print(probs[:, 1])

```

The point of the toy code is not fidelity to the full VLA stack. It is to show the structural move:

mixed data + advantage label $\implies$ better policy by conditioning.

8 Local Experimental Snapshot

To ground the concept, we built two toy examples in `recap_pi`:

- a 2D double-integrator obstacle-navigation task,
- a 3D “drone heist” task with three ordered objectives: collect artifact, activate uplink, then extract.

These are not faithful reproductions of $\pi_{0.6}^*$. They are small experiments designed to expose the exact mode-conflict RECAP is meant to resolve: the model is trained on a mixture of good and bad behavior, while the conditioned variant is prompted toward the high-advantage slice of that data.

8.1 Results Table

Experiment	Metric	Plain Imitation	RECAP Conditioned
2D obstacle navigation	Avg. effective steps	190	41

Experiment	Metric	Plain Imitation	RECAP Conditioned
2D obstacle navigation	Success rate	17%	100%
2D obstacle navigation	Collision rate	83%	0%
2D obstacle navigation	Relative success gain	1.0x baseline	6.0x higher
3D drone heist	Avg. effective steps	222	111
3D drone heist	Success rate	19%	74%
3D drone heist	Artifact completion	55%	100%
3D drone heist	Uplink activation	24%	74%
3D drone heist	Collision rate	79%	19%
3D drone heist	Relative success gain	1.0x baseline	3.88x higher

8.2 Interpretation

The 2D task demonstrates the cleanest form of the effect. Plain behavior cloning often learns an averaged policy over conflicting trajectories and crashes early, while the conditioned policy consistently takes the successful detour.

The 3D task is more interesting because it has an explicit multi-stage structure. The policy must:

1. obtain the artifact,
2. climb and route toward the uplink,
3. finish by reaching the extraction zone.

The conditioned policy does not merely improve raw path smoothness. It is much better at preserving task *sequence structure*. In other words, it is more likely to complete the right subgoals in the right order, which is exactly the type of long-horizon behavioral organization RECAP is designed to help with.

9 Figures and What They Show

9.1 2D Comparison Figure

Figure 1 is the clearest visual summary of the 2D toy result. It contains three conceptually important views:

1. **Trajectory overlays.** The plain behavior-cloned rollouts spread into the obstacle region and terminate quickly, which reflects the classic failure mode of averaging across incompatible behaviors. The RECAP-conditioned rollouts instead cluster around the clean successful corridor.
2. **Policy field slices.** The quiver plots show that the conditioned policy has learned a more coherent global action field around the obstacle structure, while the plain policy retains ambiguous or self-defeating local directions near the blocked route.
3. **Value heatmap and step-cost bars.** These panels show the mechanism behind the improvement. The value estimate assigns higher return to states that belong to the successful detour, and the timeout-penalized step metric confirms that the conditioned policy is not merely “failing faster” but actually solving the task.

Double-Integrator Navigation: Plain BC vs RECAP Advantage Prompt

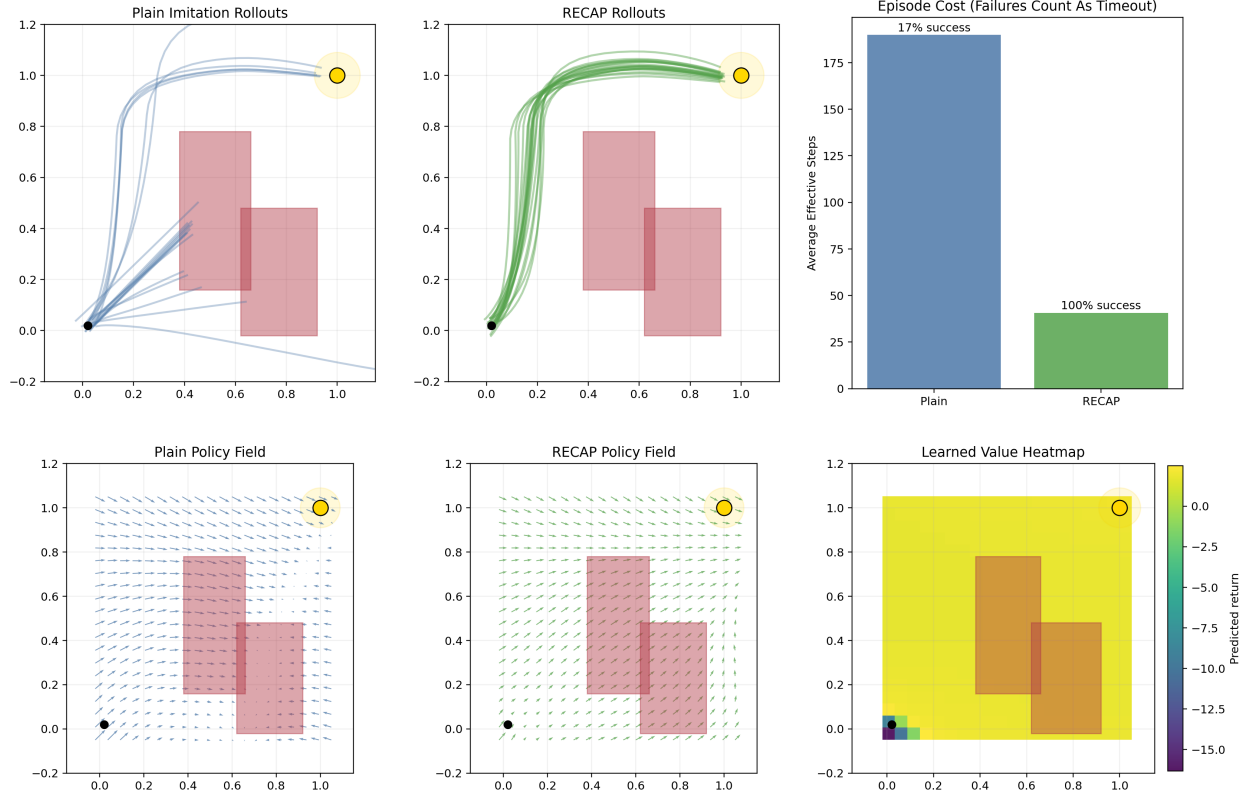


Figure 1: 2D obstacle-navigation comparison plot generated by the local toy experiment.

This figure illustrates the core RECAP idea in its simplest form: once mixed data is relabeled by advantage, the policy can be conditioned toward the successful mode rather than averaging across both success and failure.

9.2 2D Rollout Stills

Figure 4 makes the behavioral distinction more concrete. The plain policy frame shows trajectories crowding the dangerous region and dying early. The conditioned frame shows the rollouts staying organized around the same high-value path. Even in a static snapshot, the difference in mode selection is visible.

9.3 3D Drone-Heist Comparison Figure

Figure 5 is more interesting because the task has ordered subgoals rather than a single destination. The drone must collect the artifact, activate the uplink, and then extract. The figure is useful in four ways:

1. **3D trajectory panels.** The plain policy often enters blocked geometry after partial progress, whereas the conditioned policy more consistently climbs above the blocking structure and preserves the intended mission sequence.

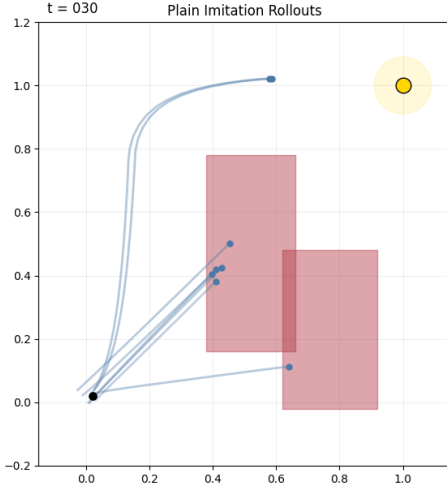


Figure 2: *

Plain imitation rollout frame

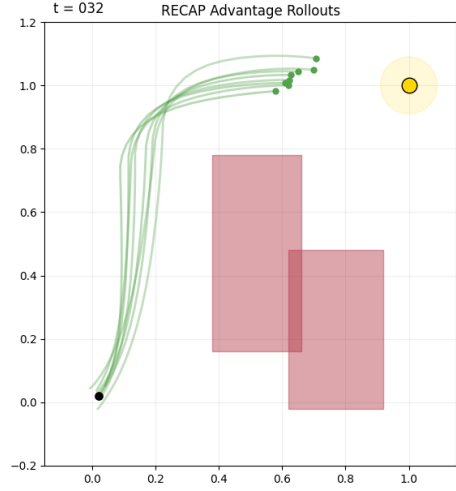


Figure 3: *

RECAP-conditioned rollout frame

Figure 4: Representative still frames extracted from the 2D rollout GIFs.

2. **Metric bars.** These show that the gain is not only in success rate. The conditioned policy improves subgoal completion rates as well, especially uplink activation, which is the middle step that depends on the policy maintaining long-horizon structure after obtaining the artifact.
3. **Projection views.** The top and side projections reveal why the 3D task is harder. The policy must choose both lateral routing and altitude management. The conditioned policy learns a cleaner high-altitude path through the bottleneck.
4. **Narrative meaning.** This figure is the local toy analogue of the RECAP story in the paper: advantage conditioning helps the model choose actions that preserve future solvability, not just immediate motion smoothness.

9.4 3D Rollout Stills

Figure 8 shows the higher-fidelity 3D animation frames. The plain policy still appears busy and locally competent, but many trajectories are pointed into hazardous regions or poorly aligned with the later subgoals. The conditioned policy frame is visually calmer: the paths are smoother, the altitude choice is more consistent, and the rollouts are better aligned with the extraction corridor after completing the earlier objectives.

The figure is useful because it highlights a subtle but important point. RECAP is not only selecting “nicer” actions. It is selecting actions that are better *because they preserve future opportunity*. That is exactly what the advantage label is supposed to encode.

10 Takeaways

Our current understanding can be summarized in five points:

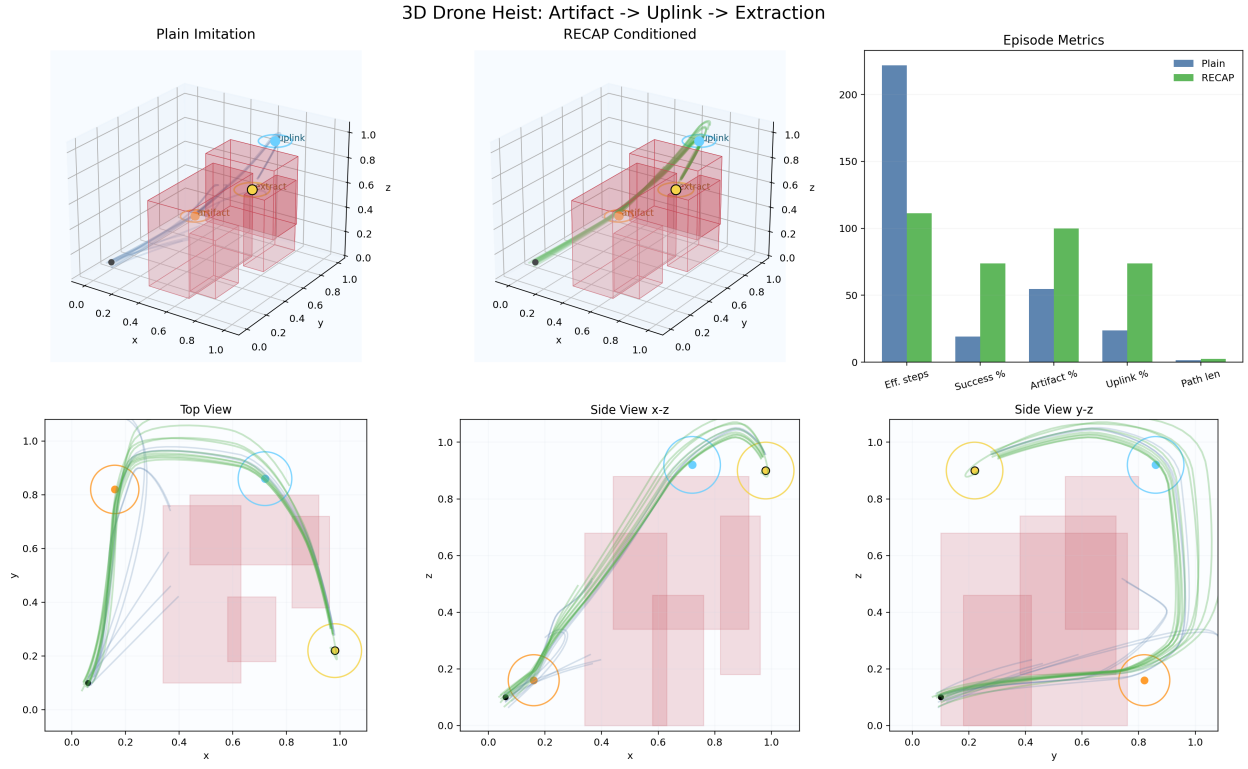


Figure 5: 3D drone-heist comparison plot generated by the local multi-stage toy experiment.

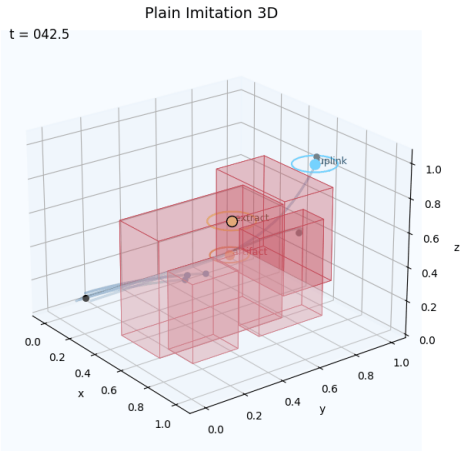


Figure 6: *
Plain imitation 3D frame

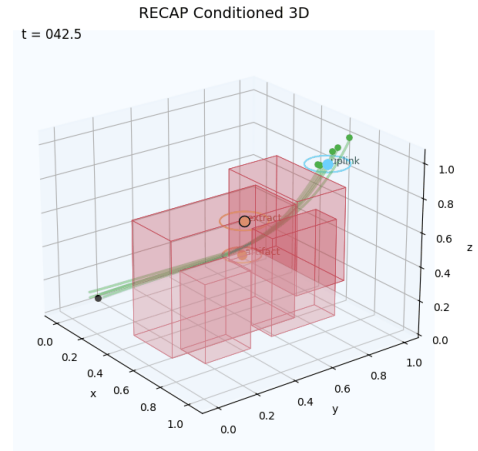


Figure 7: *
RECAP-conditioned 3D frame

Figure 8: Representative still frames extracted from the 3D GIFs.

1. RECAP reuses the semantics of advantage without doing large-scale policy-gradient RL on the main VLA.
2. The critic provides long-horizon credit assignment in a form compatible with supervised fine-tuning.
3. Binarization is important because it turns a noisy scalar signal into a robust conditioning token.
4. Human corrections fit naturally into the same framework by being marked as positive-advantage actions.
5. The resulting policy can improve over the mixed dataset simply by conditioning on the positive label at inference.

The toy results are consistent with that story. The conditioned policy is not just slightly better calibrated. It is qualitatively better at selecting the successful mode from a mixed behavioral dataset.

Artifacts

This writeup accompanies the toy examples in `recap_pi`. The current repository also contains rendered comparison plots and GIFs for both the 2D and 3D settings under `recap_pi/outputs/`.